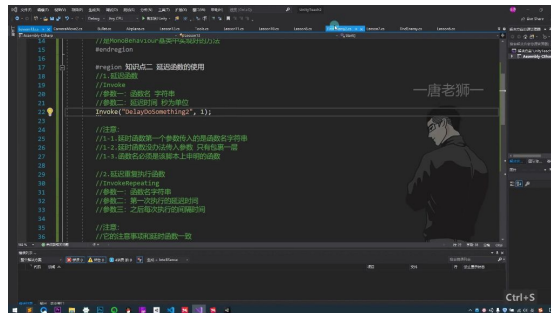


- 5秒后会执行打印"延时执行的函数"

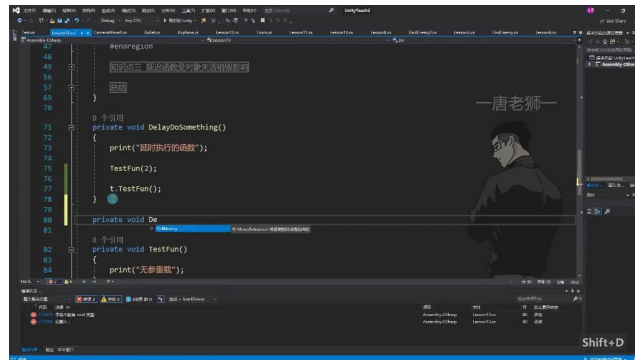
■ 注意事项:

- 函数名准确性: 必须完全匹配, 否则会报错"无法调用方法"
- 参数限制: 无法直接传递参数, 需通过包裹层间接传参
- 作用域限制: 只能调用本脚本中声明的函数



● 参数传递技巧:

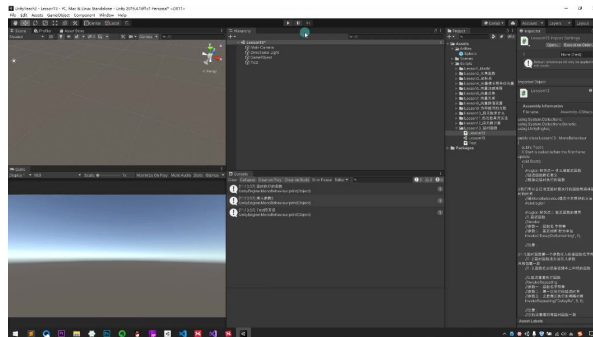
- 延迟重复执行函数InvokeRepeating 09:33



- 基本用法:

- InvokeRepeating("函数名", 首次延迟, 间隔时间)
- 示例: InvokeRepeating("DelayRe", 5, 1)表示5秒后首次执行, 之后每1秒执行一次

- 例题:InvokeRepeating使用 09:55

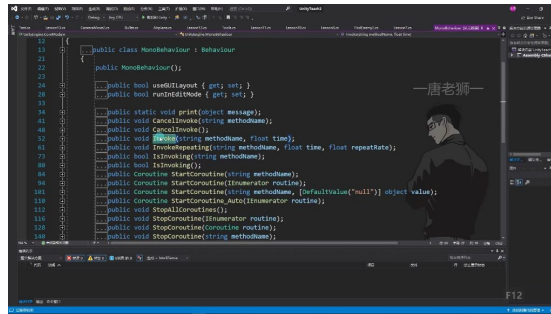


■ 实现步骤:

- 声明无参函数private void DelayRe()
- 调用InvokeRepeating("DelayRe", 5, 1)
- 运行后会先等待5秒, 然后每隔1秒重复执行

■ 共享特性:

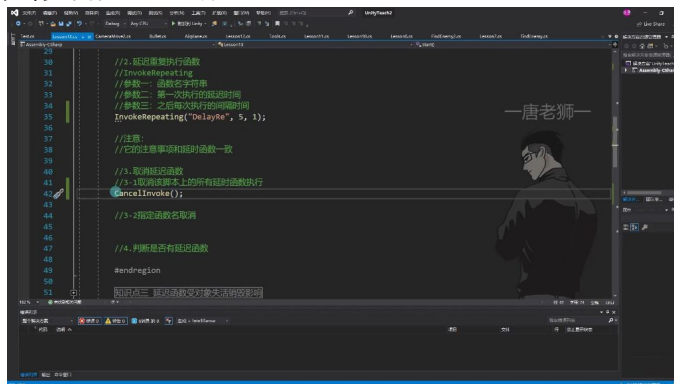
- 同样需要函数名准确
- 同样无法直接传参
- 同样只能调用本脚本函数
- 取消方法: CancelInvoke("函数名")



- **底层原理：** 这些方法都继承自MonoBehaviour基类，通过反射机制实现

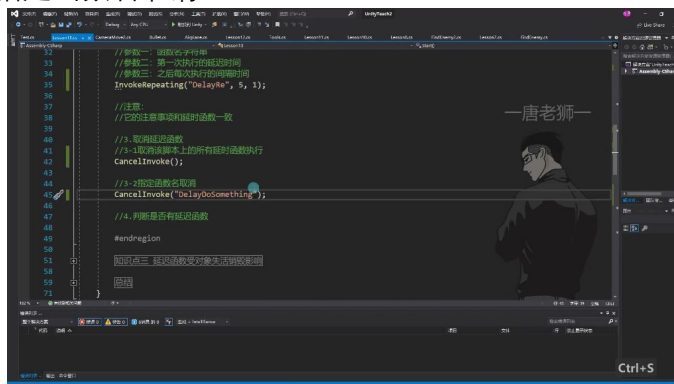
2. 取消延迟函数 12:14

1) 取消所有延迟函数 12:43



- **方法调用：** 使用CancelInvoke()可以取消当前脚本上所有正在执行的延迟函数
- **执行效果：** 调用后会立即停止所有通过Invoke或InvokeRepeating设置的延迟函数
- **使用场景：** 适用于需要完全停止所有延迟执行的情况，如游戏暂停或场景切换时
- **注意事项：** 该方法不需要任何参数，会无差别取消所有延迟函数

2) 指定函数名取消 13:33



- **方法重载：** CancelInvoke("函数名")可以取消指定名称的延迟函数
- **参数要求：** 传入字符串形式的函数名，必须与延迟函数设置的名称完全一致
- **执行特点：** 会取消该函数名的所有延迟调用，无论之前开启了多少次
- **使用示例：**
- **注意事项：**
 - 函数名必须是在当前脚本中声明的
 - 无法通过该方法取消其他脚本上的延迟函数
 - 取消操作是立即生效的

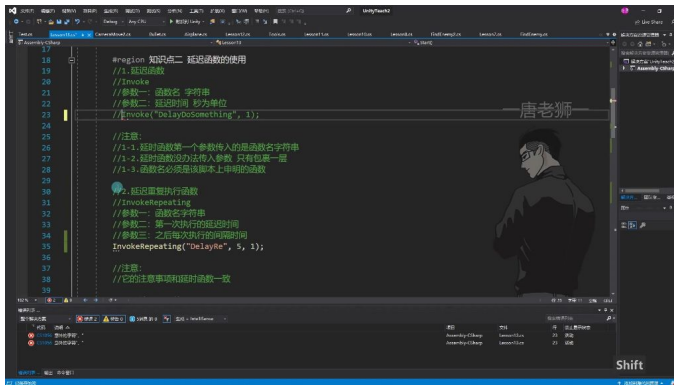
3) 判断延迟函数

- **判断方法：** 可以使用IsInvoking方法检查特定延迟函数是否正在执行
- **两种形式：**
 - IsInvoking(): 检查脚本上是否有任何延迟函数在执行

- IsInvoking("函数名"): 检查特定函数是否有延迟在执行
- **返回值:** 返回布尔值, true表示有延迟函数正在执行或等待执行

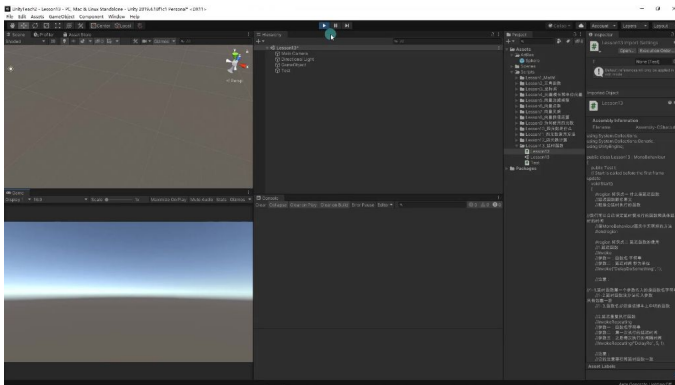
3. 延迟函数的使用

1) 基本延迟函数



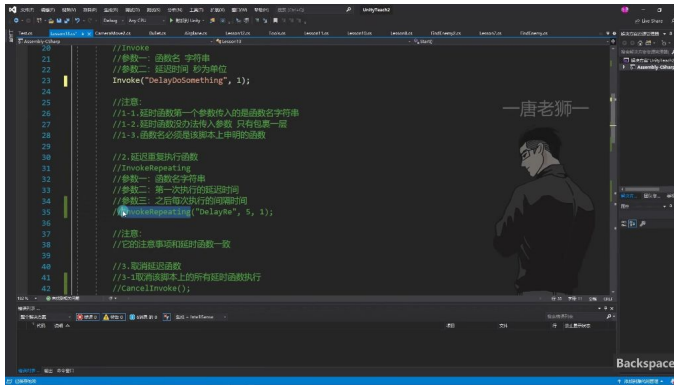
-
- **调用方式:** 使用Invoke("函数名", 延迟时间)方法
- **参数说明:**
 - 参数一: 需要延迟执行的函数名字符串
 - 参数二: 延迟时间 (秒为单位)
- **注意事项:**
 - 函数名必须为字符串形式
 - 无法直接传入参数, 需要通过包裹函数实现
 - 函数必须在该脚本中声明

2) 重复延迟函数

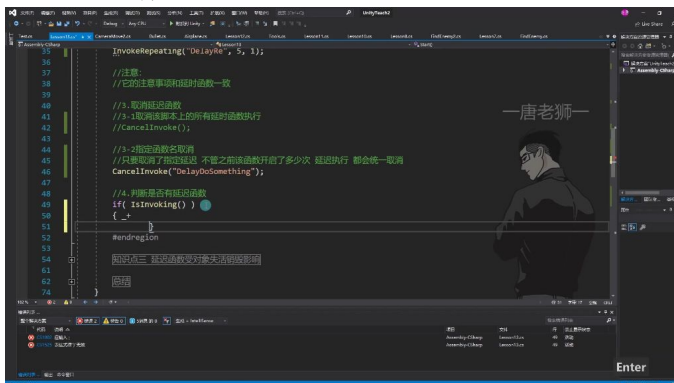


-
- **调用方式:** 使用InvokeRepeating("函数名", 首次延迟时间, 间隔时间)方法
- **参数说明:**
 - 参数一: 需要重复执行的函数名字符串
 - 参数二: 第一次执行前的延迟时间
 - 参数三: 之后每次执行的间隔时间
- **注意事项:** 与基本延迟函数一致

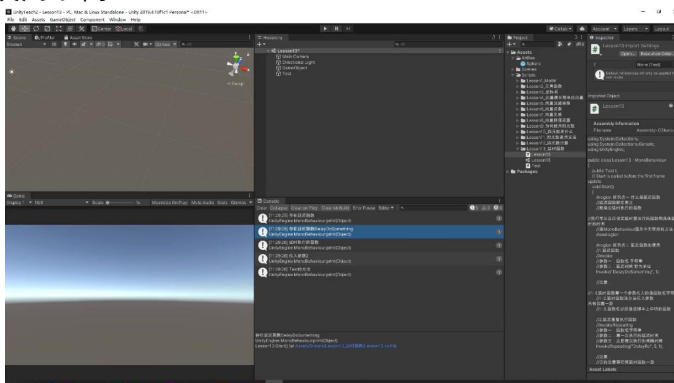
3) 取消延迟函数



- - **全部取消:** 使用CancelInvoke()取消脚本上所有延迟函数
 - **指定取消:** 使用CancelInvoke("函数名")取消特定函数的延迟执行
 - **安全特性:**
 - 取消不存在的延迟函数不会报错
 - 取消指定函数会同时取消该函数的所有延迟执行实例
- 4) 判断延迟函数



- - **全局判断:** 使用IsInvoking()判断脚本是否有任何延迟函数在执行
 - **特定判断:** 使用IsInvoking("函数名")判断特定函数是否有延迟执行
 - **返回值:** 布尔值, 配合if语句使用
 - **实际应用:**
 - 判断功能使用频率较低
 - 取消操作本身具有安全性, 无需预先判断
- 5) 使用示例



-
- **代码示例:**

```

1 // 开启延迟
2 Invoke("DelayDoSomething", 1);
3 InvokeRepeating("DelayRe", 5, 1);
4
5 // 取消延迟

```

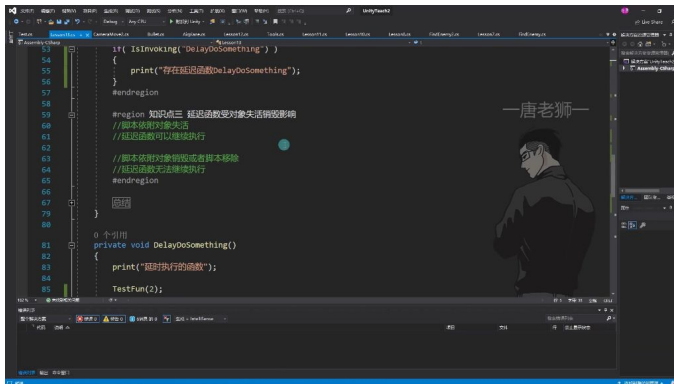
```

6 Cancellable();
7 Cancellable("DelayDoSomething");
8
9 // 判断延迟
10 if(IsInvoking()){
11     print("存在延迟函数");
12 }
13 if(IsInvoking("DelayDoSomething")){
14     print("存在DelayDoSomething延迟");
15 }

```

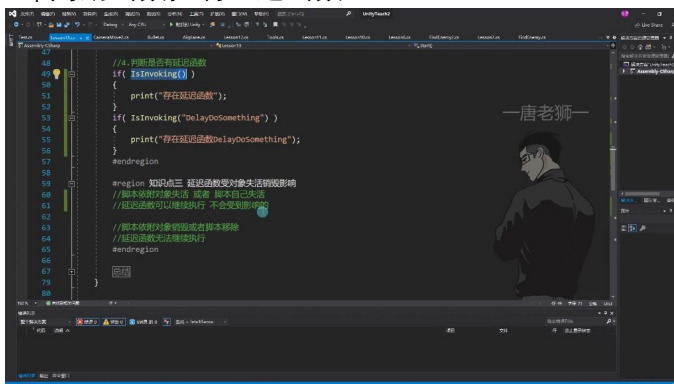
4. 延迟函数受对象失活销毁的影响 19:03

1) 对象状态对延迟函数的影响



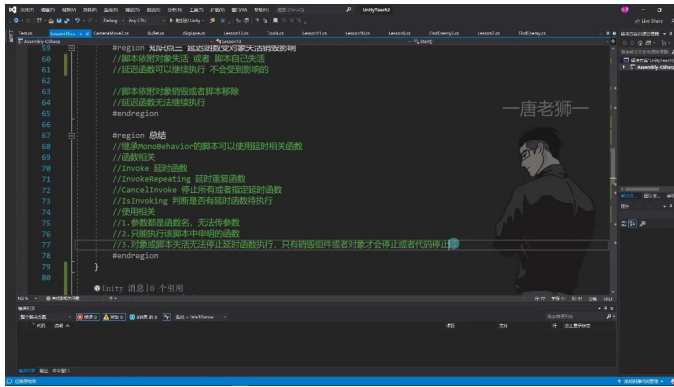
- 对象失活时:
 - 脚本依附对象失活或脚本自己失活时，延迟函数可以继续执行受影响
 - 举例：当游戏对象被设为inactive时，已启动的延迟函数仍会继续执行打印日志
- 对象销毁时:
 - 脚本依附对象销毁或脚本被移除时，延迟函数将无法继续执行
 - 举例：删除游戏对象或移除脚本组件后，延迟函数立即停止执行

2) 生命周期函数控制延迟函数



- 控制方法:
 - 在OnEnable()中开启延迟函数：对象激活时自动启动延迟执行
 - 在OnDisable()中停止延迟函数：对象失活时自动取消延迟执行
- 实现原理:
 - 利用Cancellable()在生命周期函数中精确控制延迟函数的启停
 - 弥补了对象失活不会自动停止延迟函数的特性缺陷

5. 延迟函数总结 23:47



-
- **核心API:**
 - Invoke: 单次延迟执行
 - InvokeRepeating: 重复延迟执行
 - CancelInvoke: 停止延迟执行
 - IsInvoking: 检测延迟函数状态
- **使用限制:**
 - 参数必须为函数名字符串，无法直接传参
 - 只能执行当前脚本中声明的函数
 - 需要通过包裹函数实现参数传递
- **执行特性:**
 - 不受对象/脚本失活影响
 - 会被对象销毁/脚本移除终止
 - 可通过代码主动取消执行

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
延迟函数概念	通过MonoBehaviour提供的Invoke方法，指定函数延迟执行时间	函数名需为字符串且必须存在，无法直接传参	★★
延迟函数使用	Invoke("函数名", 延迟秒数), 需声明无参函数	参数传递需包裹无参函数，仅能调用当前脚本内函数	★★★
重复延迟函数	InvokeRepeating("函数名", 首次延迟, 间隔时间)	首次执行后按间隔循环调用，取消需显式调用 CancelInvoke	★★
取消延迟函数	CancelInvoke()取消所有，CancelInvoke("函数名")取消指定	取消后无报错，即使函数未开启	★★
延迟函数状态判断	IsInvoking()检测是否有延迟函数，IsInvoking("函数名")检测指定函数	失活对象不影响执行，但销毁对象或移除脚本会终止	★★
对象状态影响	失活对象/脚本仍执行延迟函数，销毁对象或移除脚本则终止	需通过OnEnable/OnDisable生命周期手动控制启停	★★★★

封装调用外部函数	通过无参函数包裹其他对象的带参方法	无法直接跨对象调用延迟函数	★★★
----------	-------------------	---------------	-----